Universidad Católica Boliviana “San Pablo”
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

La Paz - Bolivia

GRUPO 5 S.R.L

Sistema POS para Cafetería

Integrantes:

- Claros Suntura Juan Jose *(Desarrollador FullStack)*
- Lecoña Condori Elias Milan *(Desarrollador FullStack)*
- Maldonado Carvajal Alan Ariel *(Scrum Master)*
- Torrez Calle Alvaro Ariel *(Product Owner)*

Materia: Ingenieria de Software (SIS-213)
Docente: Miguel Angel Pacheco Arteaga
Fecha: 06/03/2026
Universidad: Universidad Catolica Boliviana

Índice General

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS **1**

1.1 Introducción 1

1.2 Antecedentes 1

1.2.1 Antecedentes del objeto de estudio 1

1.2.2 Referencias técnicas de otros sistemas TPS 2

1.3 Descripción del objeto de estudio 3

1.4 Identificación del Problema 3

1.5 Formulación del Problema 3

1.6 Objetivos 4

1.6.1 Objetivo General 4

1.6.2 Objetivos Específicos 4

1.7 Justificación 4

1.7.1 Justificación Técnica 4

1.7.2 Justificación Organizacional 5

1.7.3 Justificación Económica 5

1.8 Límites y Alcances 6

1.8.1 Límites 6

1.8.2 Alcances 6

1.9 Metodología del Proyecto 7

1.9.1 Tipo de estudio 7

1.9.2 Metodología de desarrollo 7

1.9.3 Técnicas 7

1.10 Análisis preliminar del sistema TPS 7

1.11 Propuesta de solución 8

1.12 Cronograma 9

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS **11**

2.1 Sistemas de Información Organizacional 11

2.1.1 Definición 11
2.1.2 Componentes y Tipos 11
2.2 Sistema de Procesamiento de Transacciones (TPS) 11
2.3 Arquitectura de sistemas web 12
2.4 Seguridad en sistemas de información 12
2.5 Base de datos 13
2.6 Metodología de desarrollo 14

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS 15
3.1 Análisis del sistema 15
3.2 Determinación de requerimientos 16
3.2.1 Requerimientos funcionales 16
3.2.2 Requerimientos no funcionales 16
3.3 Modelado del sistema 17
3.3.1 Historias de Usuario 17
3.3.2 Diagramas UML 17
3.4 Diseño del sistema 18
3.4.1 Arquitectura del sistema 18
3.5 Diseño de la Base de Datos 18
3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA 19
3.6.1 Módulo de Catálogo, Menú y Mesas 19
3.6.2 Módulo de Usuarios y Roles 19
3.6.3 Módulo de Transacciones (El Punto de Venta) 20
3.6.4 Módulo de Reportes 20
3.7 Capa Backend Funcional 20
3.8 Validación y pruebas del sistema 21
3.9 Desarrollo del prototipo funcional 21
3.10 Documentación de Ingeniería Completa 21

Capítulo 4. CONCLUSIONES Y RECOMENDACIONES 23
4.1 Conclusiones 23

4.2 Recomendaciones 23

Índice de Figuras, Tablas y Diagramas

Figuras

1 Ejemplo de prueba en el análisis del sistema 15

Tablas

Diagramas

1 Ejemplo general de Diagrama Estructural UML 17

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS

1.1 Introducción

Los Sistemas de Información Organizacional cumplen un rol fundamental en la actualidad, permitiendo a las empresas gestionar sus operaciones de manera eficiente y centralizada. En este contexto, la evolución de los sistemas de Procesamiento de Transacciones (TPS, por sus siglas en inglés) ha sido clave para la automatización de procesos operativos rutinarios, garantizando la consistencia y seguridad de la información.

La necesidad de automatización de procesos se hace evidente ante el crecimiento de las transacciones diarias en cualquier organización moderna, donde el registro manual desencadena errores humanos, pérdida de tiempo y falta de trazabilidad. El uso de sistemas web organizacionales centraliza esta gestión, permitiendo el acceso concurrente desde diferentes ubicaciones y dispositivos de forma segura.

Ejemplo de enfoque: Este proyecto consiste en el desarrollo de un Sistema de Información Organizacional Web basado en la evolución del enfoque TPS, enfocado en un sistema de Punto de Venta (POS) para una cafetería. Este permitirá gestionar el flujo de pedidos, usuarios, catálogo de productos, asignación de mesas, cobros y reportes mediante una plataforma tecnológica accesible y segura.

1.2 Antecedentes

1.2.1 Antecedentes del objeto de estudio

Actualmente, muchas cafeterías a nivel local llevan a cabo sus procesos de manera 100% manual. Esta práctica genera inconsistencia en los datos financieros, problemas de comunicación entre la caja y el área de preparación, y cuellos de botella en las horas de mayor afluencia.

Ejemplos de problemas existentes:

- Registro manual de comandas en libretas de papel, lo que provoca errores, pér-

didas o ilegibilidad.

- Falta de control granular de usuarios (no se puede identificar qué cajero procesó o anuló una orden).
- Cobro basado en cálculo mental o calculadoras simples, generando descuadres de caja.
- Nula trazabilidad de las transacciones diarias (no hay base de datos histórica).

1.2.2 Referencias técnicas de otros sistemas TPS

En el contexto local de La Paz, Bolivia, existe una notable carencia de sistemas TPS especializados y accesibles para el rubro de las cafeterías, o en su defecto, hay un desconocimiento generalizado sobre soluciones de *software* a medida. Por lo tanto, como Ingeniero de Requerimientos y Datos, el análisis de referencia se basa en el modelo transaccional "tradicional" o empírico que emplean actualmente la mayoría de las cafeterías en la ciudad sin un sistema digital centralizado:

1. **Sistema analógico de comandas y caja básica**
    - **Tipo de sistema:** TPS manual y físico.
    - **Funcionalidades principales:** El mesero anota el pedido y el número de mesa en una libreta de papel (comanda), el cual se entrega físicamente a la barra de preparación. El cobro se realiza calculando mentalmente o con una calculadora de escritorio, guardando el dinero en una caja registradora simple o gaveta de efectivo.
    - **Diferencia con el sistema propuesto:** Este método empírico carece de cualquier tipo de integridad de datos. Nuestro sistema de *software* digitalizará la entrada de la orden y la asignación de mesas en tiempo real, eliminando errores de cálculo, previniendo la pérdida de comandas de papel y asegurando que cada producto despachado quede registrado inmutablemente en la base de datos para un arqueo de caja exacto.
2. **Registro diferido en hojas de cálculo (Excel)**
    - **Tipo de sistema:** TPS semiautomatizado y no concurrente.

- **Funcionalidades principales:** Al final de la jornada, el administrador recolecta los apuntes manuales e intenta cuadrar los ingresos del día transcribiéndolos a una hoja de cálculo.
- **Diferencia con el sistema propuesto:** Una hoja de cálculo no es una base de datos transaccional ni opera en tiempo real. El sistema web propuesto validará cada transacción en el momento exacto del cobro de la mesa, guardando relaciones seguras y automáticas entre el cajero en turno, los productos vendidos y la fecha exacta de emisión.

1.3 Descripción del objeto de estudio

El proyecto se enmarca dentro de una cafetería que requiere controlar el flujo diario de sus operaciones de venta rápida. Los procesos organizacionales principales incluyen la toma de órdenes por mesa, el cobro exacto y la auditoría de ventas.

- **Actores del sistema:** Administradores y Cajeros (Operadores POS).
- **Flujo general del negocio:** Un cajero registra una orden en la interfaz asignándola a una mesa, cobra al cliente, y el sistema consolida la transacción inmutablemente en la base de datos.

1.4 Identificación del Problema

La cafetería presenta dificultades en la gestión rápida de pedidos, control de usuarios operacionales, registro de pagos y generación de reportes financieros debido a la ausencia de un motor de base de datos automatizado y centralizado. Esto incrementa los tiempos de espera del cliente, dificulta los arqueos de caja al final del turno y aumenta exponencialmente los márgenes de error operativo por sumas manuales.

1.5 Formulación del Problema

¿Cómo desarrollar un Sistema de Información Organizacional Web (POS) basado en el enfoque TPS para una cafetería, que permita gestionar pedidos, mesas, usuarios,

transacciones y reportes de manera eficiente y criptográficamente segura?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un Sistema de Punto de Venta (POS) Web basado en el enfoque de Procesamiento de Transacciones (TPS) que permita gestionar el flujo operativo de una cafetería mediante una arquitectura *backend* funcional y persistencia de datos estructurada.

1.6.2 Objetivos Específicos

- Analizar los procesos actuales de la cafetería y el flujo de comunicación manual.
- Diseñar la base de datos orientada a documentos para optimizar la velocidad del carrito de compras.
- Implementar el módulo de usuarios aplicando encriptación y control de acceso por roles (JWT).
- Implementar el módulo de gestión del catálogo de productos y mesas.
- Implementar el módulo transaccional (POS) garantizando la integridad ACID en las ventas.
- Implementar el módulo de reportes y auditoría transaccional.
- Desarrollar la capa *backend* funcional consumible a través de una API REST.
- Validar el sistema mediante pruebas de *software* estructuradas.
- Documentar integralmente el esquema de datos y el código fuente.

1.7 Justificación

1.7.1 Justificación Técnica

Desde el rol de ingeniería de requerimientos y datos, la implementación de este sistema en una cafetería que opera de forma 100% manual (con libretas de papel y sin

ningún tipo de *software*) se justifica por la necesidad urgente de establecer un núcleo de persistencia de datos robusto y seguro. Se diseñará una base de datos estructurada (adoptando el paradigma de la pila MERN, como MongoDB) que reemplace la vulnerabilidad de los cuadernos físicos. Esto garantiza la integridad referencial de la información, conectando de forma exacta el catálogo de productos con el carrito de ventas, el número de mesa y el usuario que procesa el cobro. El *backend* (Node.js/Express) manejará de forma atómica cada transacción, mientras que la seguridad se blindará mediante encriptación de contraseñas y autenticación por *tokens* (JWT), asegurando la protección de los datos financieros ante cualquier interrupción o alta concurrencia.

1.7.2 Justificación Organizacional

Actualmente, el uso exclusivo de comandas escritas a mano y la comunicación verbal generan un caos organizativo constante: los meseros equivocan las mesas, los pedidos se pierden o resultan ilegibles para el área de preparación, y no hay control sobre quién realiza qué cobro. La implementación de este TPS estandarizará el flujo operativo. A nivel de requerimientos, el sistema forzará jerarquías claras de acceso (Administrador y Cajero) y digitalizará la selección de productos y la asignación específica de las mesas. Esto elimina de raíz la asimetría de información, asegurando que el estado de cada orden, desde que se toma el pedido en la mesa hasta que se imprime el ticket final, sea transparente y rastreable en tiempo real para todo el personal autorizado.

1.7.3 Justificación Económica

La dependencia del cálculo mental y de anotaciones manuales para cobrar a los clientes y realizar el arqueo de caja es altamente susceptible al error humano. Estas fallas diarias se traducen en cobros incompletos, descuadres de caja y una pérdida monetaria silenciosa y constante para la cafetería. Al transicionar a un motor transaccional de Punto de Venta (POS) centralizado —que calcula automáticamente los subtotales, impuestos y totales finales antes de generar la factura— se erradican las fugas de capital por malas sumas. Además, la centralización de los datos permitirá a la gerencia con-

sultar reportes históricos exactos (como los productos más vendidos), facilitando una toma de decisiones inteligente y la optimización del presupuesto para la compra de insumos.

1.8 Límites y Alcances

1.8.1 Límites

El sistema:

- Será exclusivamente web (no incluirá aplicaciones móviles nativas).
- Usará una base de datos NoSQL orientada a documentos enfocada en transacciones rápidas.
- Tendrá un sistema cerrado de autenticación de usuarios locales (no integrará inicios de sesión sociales).
- No incluirá integración directa a pasarelas de pago bancarias físicas en su primera versión (los pagos se registrarán de forma lógica).

1.8.2 Alcances

El sistema permitirá:

- Gestionar el catálogo interactivo de la cafetería (categorías, productos, precios).
- Gestionar usuarios, asignar roles y administrar permisos granulares mediante *middlewares*.
- Registrar transacciones en el POS vinculando carritos de compra a mesas específicas.
- Generar reportes tabulares y calcular cortes de caja.

1.9 Metodología del Proyecto

1.9.1 Tipo de estudio

Por su naturaleza, la investigación será de tipo **Aplicado** (orientado a la solución de un problema concreto), **Tecnológico** (mediante el uso de herramientas de *software*) y **Descriptivo** (para modelar el escenario y comportamiento actual).

1.9.2 Metodología de desarrollo

Se aplicará la metodología ágil ***Scrum***, que se adapta idealmente al desarrollo de *software* iterativo e incremental:

- ***Sprints*** (Iteraciones): Ciclos de trabajo de 2 a 4 semanas.
- ***Product Backlog*** (Pila de producto): Lista priorizada de todos los requerimientos y módulos del sistema.
- ***Sprint Backlog*** (Pila del ciclo): Tareas seleccionadas para ser desarrolladas en el ***Sprint*** actual.
- **Entregables:** Prototipos funcionales al final de cada iteración demostrando valor.

1.9.3 Técnicas

- **Entrevistas:** Al personal clave de la organización para levantar requerimientos.
- **Observación directa:** De los procesos manuales actuales en el sitio.
- **Modelado de Datos:** Estructuración de colecciones JSON y flujos de API REST.
- **Modelado UML:** Para representar gráficamente el diseño de la solución (casos de uso, diagramas de clase, etc.).

1.10 Análisis preliminar del sistema TPS

Como Ingeniero de Requerimientos y Datos, el análisis preliminar dictamina que la transición de una cafetería puramente manual a un entorno digital requiere modelar transacciones rápidas y precisas, integrando la gestión física del local (mesas). Se han

especificado los siguientes requerimientos de persistencia y flujo:

- **Detalle de Usuarios (Actores y Control de Acceso):**
  - **Administrador:** Actor con privilegios absolutos (*Full CRUD*). Su rol a nivel de datos implica alimentar las entidades maestras del sistema: gestión del catálogo de ítems de la cafetería (nombres, precios, imágenes, categorías como 'Cafés', 'Postres'), control de inventario base y la creación o inhabilitación de cuentas para el personal operativo.
  - **Cajero / Operador POS:** Actor confinado a la interfaz de Punto de Venta. Su interacción con la base de datos es de lectura sobre el menú y de escritura intensiva sobre las colecciones/tablas de ventas, sin permisos para alterar precios históricos o borrar registros contables.
- **Detalle de Transacciones (El Flujo de la Orden y Mesas):**
  - **Transacción POS y Asignación:** Reemplazando por completo la libreta de papel, el nuevo flujo demanda que el cajero construya la orden mediante un panel interactivo con categorías. El modelo de datos requerirá que, antes de proceder al cobro, la colección del carrito de compras (compuesta por los ítems seleccionados y sus cantidades) sea vinculada obligatoriamente al nombre del cliente y al **número de mesa**.
  - **Facturación y Cierre:** Una vez que se confirma el método de pago, el *backend* debe procesar matemáticamente el subtotal, aplicar los impuestos correspondientes y sellar la transacción con la fecha exacta. El registro resultante en la base de datos se vuelve inmutable y dispara en el *frontend* la opción de generar e imprimir la factura o ticket (*Bill/Receipt*), dando por finalizado el servicio para esa mesa.

1.11 Propuesta de solución

- **Sistema:** Sistema de Punto de Venta (POS) Web enfocado en TPS.
- **Arquitectura:** Patrón Cliente — Servidor con separación de responsabilidades API REST.

Tecnologías sugeridas (Pila MERN):

- ***Backend*:** Entorno Node.js con el framework Express.
- **Base de datos:** Motor NoSQL MongoDB (orientado a documentos).
- ***Frontend*** (Interfaz): Librería React con Redux para manejo de estados.

1.12 Cronograma

El despliegue del proyecto abarca un tiempo estimado de desarrollo según las siguientes fases:

Fases	Entregables / Hitos	Duración
Análisis	Captura de requerimientos, diseño UI/UX y esquema de colecciones (DB).	2 Semanas
Diseño	Arquitectura tecnológica, modelado UML y definición técnica de API.	2 Semanas
Desarrollo	Programación *backend*, *frontend* y construcción de la base de datos.	6 Semanas
Pruebas	Ejecución de validaciones transaccionales y de estrés.	2 Semanas
Documentación	Finalización de manuales y armado del documento de grado.	2 Semanas

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS

2.1 Sistemas de Información Organizacional

2.1.1 Definición

Un sistema de información organizacional es un conjunto de componentes interrelacionados que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, el control y la coordinación dentro de una entidad.

2.1.2 Componentes y Tipos

Involucran equipo (*hardware*), programas (*software*), bases de datos, redes, procedimientos y recursos humanos. Existen diferentes tipos, como los Sistemas de Soporte a Decisiones (DSS), de Información Gerencial (MIS) y los Sistemas de Procesamiento de Transacciones (TPS).

2.2 Sistema de Procesamiento de Transacciones (TPS)

El TPS es la columna vertebral de cualquier sistema organizacional que recolecta y procesa las transacciones generadas en el día a día.

- **Características principales:** Procesamiento rápido, alta confiabilidad, capacidad para manejar gran volumen de datos de forma estandarizada y estricta integridad en cada transacción (especialmente en Puntos de Venta).
- **Funciones:** Captura de datos de origen (carrito de compras), verificación matemática, procesamiento inmediato, actualización de bases de datos maestras y emisión de comprobantes/tickets.
- **Evolución hacia sistemas web:** Han pasado de terminales cerradas monolíticas a integrarse mediante la web, lo cual otorga ubicuidad y permite operaciones distribuidas.

2.3 Arquitectura de sistemas web

- **Cliente (*Frontend*):** La interfaz que interactúa con el usuario final a través de un navegador web, responsable de la presentación de la información (Ej. Pantalla POS de React).
- **Servidor (*Backend*):** La computadora central o contenedor de la nube que procesa la lógica de negocios, gestionando la concurrencia y los cálculos matemáticos.
- **API (*Application Programming Interface*):** El puente de comunicación documentado entre el Cliente y el Servidor (vía JSON).
- **Base de datos:** Repositorio central donde reposa la persistencia de las entidades.

2.4 Seguridad en sistemas de información

Como modelador y encargado de la seguridad arquitectónica, se establece que un sistema de ventas (POS) debe proteger de forma absoluta sus *endpoints* (rutas de API) y la persistencia de datos.

- **Autenticación mediante JWT:** Se descartan las sesiones tradicionales. El sistema implementa JSON Web Tokens (JWT). Tras validar credenciales (contraseñas previamente procesadas con funciones criptográficas unidireccionales de *hash*, como *bcrypt*), el *backend* emite un token firmado. Este token viaja en las cabeceras HTTP de cada petición del cliente, garantizando que el usuario es quien dice ser sin consultar la base de datos reiteradamente.
- **Roles y Autorización:** El modelo de datos incluye una propiedad rígida de “Rol” (ej. Administrador, Cajero). La autorización se ejecuta mediante *Middlewares* (bloques de código intermedios en el *backend*) que desencriptan el *payload* del token y rechazan con un error 403 (Prohibido) cualquier intento de un Cajero de acceder a las rutas de eliminación de usuarios o reportes gerenciales.
- **Control de acceso:** Tanto a nivel de la interfaz (ocultando botones de configu-

ración a cajeros) como a nivel de capa de datos, se aplican políticas estrictas para evitar inyecciones maliciosas o robo de sesiones, blindando el flujo desde que se presiona “Cobrar” hasta que la información reposa en el disco.

2.5 Base de datos

El diseño de la persistencia de datos constituye el corazón del sistema, siendo responsabilidad directa de la ingeniería de datos modelar la información de la cafetería para que sea escalable, rápida y matemáticamente exacta. Basado en la arquitectura MERN (MongoDB, Express, React, Node.js), se adopta un enfoque orientado a documentos que optimiza la velocidad transaccional en el Punto de Venta:

- **Modelo Documental (NoSQL) y Referencias Lógicas:** Al utilizar MongoDB, la base de datos abandona las tablas rígidas en favor de colecciones de documentos JSON (BSON). Sin embargo, los principios lógicos de integridad se mantienen ineludibles mediante el uso de esquemas de validación estrictos (como *Mongoose*). Las colecciones maestras se identifican y separan claramente: `Usuarios` (Personal operativo), `Categorías` (Clasificación del menú), `Productos` (Ítems de venta), `Mesas` (Gestión del espacio físico) y `Facturas/Órdenes` (*Bills* - Registro de la transacción). Se definen referencias explícitas (a través de *ObjectIds*) entre ellas para establecer la cardinalidad (ej. una Orden está vinculada unívocamente a un Cajero y asignada a una Mesa específica).
- **Desnormalización Controlada y Estructura del Carrito:** A diferencia de un modelo relacional tradicional, MongoDB aprovecha la desnormalización para mejorar el rendimiento de lectura. Por requerimientos de diseño de un POS, se realiza una desnormalización controlada en las `Órdenes`: al registrar una venta, no solo se guarda la referencia del producto, sino que se incrusta el arreglo completo del carrito (`cartItems`). El nombre de la bebida, sus modificadores (ej. tamaño, adiciones) y su **precio exacto en ese instante** se copian dentro del documento de la orden. Esto garantiza que la información histórica sea inmutable frente a futuros cambios de costos en el catálogo.

- **Transacciones Multi-documento y Aislamiento (ACID):** En el entorno TPS, una transacción es indivisible. Registrar una venta en la cafetería implica una cadena de eventos: calcular totales, insertar el documento de la factura, asociar el método de pago y actualizar el estado de disponibilidad de la `Mesa`. El motor de MongoDB se configura utilizando *Sesiones de Transacción* para garantizar las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) a través de múltiples colecciones. Esto asegura que si ocurre un fallo de red o desconexión a la mitad del proceso de cobro, la base de datos ejecute un *Rollback* (reversión completa), previniendo que existan "ventas a medias" o corrupciones en los arqueos de caja.

2.6 Metodología de desarrollo

Scrum Es un marco de trabajo colaborativo que permite abordar proyectos complejos.

- **Roles:** ***Product Owner*** (Dueño del producto), ***Scrum Master*** (Facilitador) y ***Developers*** (Equipo de desarrollo).
- **Artefactos:** ***Product Backlog*** (Pila de producto), ***Sprint Backlog*** (Pila del ciclo) e Incremento del Producto.
- **Eventos:** ***Sprint Planning*** (Planificación), ***Daily Scrum*** (Reunión diaria), ***Sprint Review*** (Revisión) y ***Sprint Retrospective*** (Retrospectiva).

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS

3.1 Análisis del sistema

En base a la recopilación de datos, la estructura organizacional del sistema de la cafetería identifica actores con responsabilidades asimétricas: Administradores, enfocados en la gobernanza del catálogo y métricas; y Cajeros, cuyo objetivo es el alto rendimiento en el registro y cobro de pedidos. El flujo exige digitalizar cada interacción, desde la selección del producto táctil hasta su asignación a la mesa y el cobro final.

Figura 1: Ejemplo de prueba en el análisis del sistema

3.2 Determinación de requerimientos

3.2.1 Requerimientos funcionales

Los requerimientos funcionales expresan lo que el sistema **debe hacer** operativamente.

- **RF01:** El sistema permitirá autenticar de forma segura a cajeros y administradores.
- **RF02:** El sistema proporcionará una interfaz de POS para la selección ágil de productos del menú por categoría.
- **RF03:** El sistema permitirá asignar obligatoriamente un número de “Mesa” a cada carrito de compra.
- **RF04:** El sistema calculará automáticamente subtotales, impuestos y totales conforme se añaden o restan productos.
- **RF05:** El sistema permitirá a los administradores gestionar colecciones maestras (Productos, Categorías, Usuarios).
- **RF06:** El sistema generará e imprimirá comprobantes (tickets) al concretar el cobro.

3.2.2 Requerimientos no funcionales

Establecen las restricciones y la forma en cómo debe operar y comportarse estructuralmente la aplicación.

- **Seguridad:** Encriptación de contraseñas usando algoritmos seguros (*bcrypt*) y protección de rutas con JWT.
- **Rendimiento:** Tiempos de procesamiento del carrito y confirmación de pago menores a 1 segundo para no detener la fila.
- **Usabilidad:** Interfaz basada en principios POS: botones amplios e interacciones con mínimos clics.
- **Escalabilidad:** Separación modular estricta entre *Frontend* (React) y *Backend* (Node.js).

3.3 Modelado del sistema

3.3.1 Historias de Usuario

Se definen detallando la necesidad y la regla de aceptación comercial:

- *Como* administrador de la cafetería, *quiero* gestionar los productos y sus precios *para* mantener el menú actualizado.
- *Como* cajero en turno, *quiero* agregar bebidas al ticket y asignarlo a una mesa mediante la pantalla *para* atender ágilmente a los clientes y evitar confusiones en sala.

3.3.2 Diagramas UML

A continuación se muestra un ejemplo genérico de la estructura de un diagrama, en este caso, se deben incrustar aquí los diagramas correspondientes: Diagramas de Casos de Uso, Clases/Esquemas NoSQL, y Actividades.

Diagrama 1: Ejemplo general de Diagrama Estructural UML

3.4 Diseño del sistema

3.4.1 Arquitectura del sistema

Modelo: Arquitectura Web Cliente-Servidor (Pila MERN) El sistema se dividirá lógicamente en una aplicación cliente *Single Page Application* (React) interactuando asíncronamente con un servidor *backend* (Node.js/Express) que expondrá puntos de enlace (*endpoints*) REST y conectará con el motor de base de datos.

3.5 Diseño de la Base de Datos

Se diseña bajo el paradigma documental (NoSQL) garantizando el aislamiento financiero. Las restricciones en el esquema aseguran que una “Orden” sea indivisible e incorpore el carrito completo para inmutabilidad histórica.

Campo (Documento JSON)	Tipo	Validaciones / Vínculos	Descripción
`_id`	ObjectId	Único (PK lógica)	Identificador nativo de MongoDB para la factura.
`usuario_id`	ObjectId	Ref: ‘Usuario’	ID del cajero responsable de cobrar la orden.
`mesa_asignada`	String/Int	Requerido	Mesa física vinculada al pedido.

Campo (Documento JSON)	Tipo	Validaciones / Vínculos	Descripción
`cartItems`	Array	Contiene Objetos	Arreglo desnormalizado con los productos, cantidades y precios exactos en el momento de venta.
`total_pagado`	Number	Mínimo 0	Monto económico final calculado y liquidado.

3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA

3.6.1 Módulo de Catálogo, Menú y Mesas

Módulo fundamental para configurar el entorno de la cafetería.

- **Gestión de colecciones:** Interfaces CRUD para ingresar categorías, ítems del menú con sus precios, y la estructura de las mesas del local.
- **Estados de visualización:** Activación y desactivación de productos agotados sin borrarlos de la base de datos.

3.6.2 Módulo de Usuarios y Roles

La barrera de seguridad criptográfica del sistema TPS.

- **Filtro Criptográfico:** Validaciones de contraseña cruzadas con *bcrypt* en el inicio de sesión.

- **Control de acceso (RBAC):** El *frontend* renderizará la “Terminal POS” para los cajeros y bloqueará las pestañas de “Administración”, regla que es respaldada por los *middlewares* de validación de *tokens* JWT en el servidor.

3.6.3 Módulo de Transacciones (El Punto de Venta)

Núcleo central del objeto TPS, diseñado para alta velocidad operativa.

- **Interfaz de Venta:** Pantalla con cuadrícula de productos a la izquierda y el carrito reactivo a la derecha.
- **Motor Matemático:** Sumatorias dinámicas (Redux) inyectables a la API para certificar la venta atómica.
- **Cobro y Cierre:** Ventana modal que captura el método de pago, asigna la mesa, y consolida irrevocablemente el documento JSON en MongoDB, disparando la generación del ticket.

3.6.4 Módulo de Reportes

Módulo analítico que destila la información transaccional operativa del local.

- Extracción de ingresos consolidados mediante consultas de agregación (*Aggregations*) en MongoDB.
- Filtros por usuario y fechas para realizar el cierre y arqueo de caja exacto al final del turno.

3.7 Capa Backend Funcional

El *backend* es el cerebro transaccional aislado de la interfaz gráfica, diseñado bajo principios REST:

- **Controladores (*Controllers*):** Capturan las peticiones HTTP (req, res).
- **Modelos/Esquemas (*Mongoose Models*):** Representación estricta de la estructura de los documentos en la base de datos (Ej. `BillSchema`, `ItemSchema`).

- **Capa de Seguridad (*Middlewares*):** Componentes (como `verifyToken`) que inspeccionan las cabeceras HTTP antes de conceder cualquier inserción a la base de datos.
- **Lógica Transaccional:** Uso de sesiones para asegurar que el guardado de la factura y la limpieza de la mesa se ejecuten como una transacción ACID única.

3.8 Validación y pruebas del sistema

El sistema asegura la calidad del producto final a través de distintas evaluaciones:

- **Pruebas de Estrés Unitarias:** Validar que el servidor sume grandes carritos de compra sin errores de formato decimal.
- **Pruebas funcionales de reversión:** Simular el error de la red justo en el momento de procesar un pago para verificar que la transacción se deshaga (*Rollback*) correctamente.
- **Pruebas de aceptación:** Pruebas finales en la caja registradora física para certificar la fluidez del *software*.

3.9 Desarrollo del prototipo funcional

A lo largo de los *sprints* se generan prototipos incrementales exponiendo sus interfaces interactivas reflejando el caso de éxito: desde el *login* del cajero, armado de la orden, selección de mesa, hasta el cobro. *(Incluir evidencias y capturas de pantalla reales aquí)*.

3.10 Documentación de Ingeniería Completa

Se acompañan los anexos técnicos:

- **Documentación funcional:** Relevamiento documentado de requerimientos e historias de usuario de la cafetería.
- **Documentación técnica:** Modelo arquitectónico, diseño de esquemas BSON (Colecciones) y documentación de *endpoints* (API REST).

- **Documentación del sistema:** Manual de usuario, variables de entorno y directrices de despliegue.

Capítulo 4. CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

- Tras el proceso de desarrollo, se implementó satisfactoriamente el Sistema de Punto de Venta (POS) Web basado en el enfoque TPS para la cafetería, abordando con éxito el reemplazo de comandas manuales por un entorno digital unificado.
- La adopción de la pila MERN (MongoDB, Express, React, Node.js) y el diseño de persistencia orientada a documentos demostraron ser ideales para optimizar la velocidad de escritura del carrito de compras y garantizar la inmutabilidad de los reportes históricos.
- La seguridad del sistema fue robustecida a través del esquema de autenticación JWT y encriptación de credenciales, consolidando un TPS que erradica los descuadres de caja y facilita la gestión de mesas y auditorías administrativas.

4.2 Recomendaciones

- **Mejoras futuras y Escalabilidad:** Es recomendable planificar la ampliación del módulo transaccional para descontar automáticamente ingredientes de un inventario maestro (ej. gramos de café, litros de leche) cada vez que se cobre un producto.
- **Seguridad y Mantenimiento:** Monitorear activamente los registros del servidor, rotar periódicamente la clave secreta de firmado de los *tokens* JWT y actualizar las dependencias de Node.js para prevenir vulnerabilidades.
- **Migración a la Nube:** Aprovechando la naturaleza NoSQL de MongoDB, se recomienda a futuro desplegar la base de datos en clústeres administrados (como MongoDB Atlas) para asegurar redundancia, *backups* automáticos y la posibilidad de conectar múltiples sucursales de la cafetería.